2. DAG 구조와 인증서 시스템

2.1 DAG (Directed Acyclic Graph) 개념

2.1.1 기본 구조

DAG는 Narwhal의 핵심 데이터 구조로, 인증서(Certificate) 간의 의존성을 나타내는 방향성 비순환 그래프입니다.

```
// DAG의 메모리 표현

pub type Dag = HashMap<Round, HashMap<PublicKey, (CertificateDigest, Certificat
```

2.1.2 DAG의 특징

- **라운드 기반**: 각 라운드마다 노드들이 인증서 생성
- **부모 참조**: 각 인증서는 이전 라운드의 인증서들을 부모로 참조
- **비동기성**: 노드들이 독립적으로 인증서 생성 가능
- **인과관계**: 부모-자식 관계로 트랜잭션 순서 결정

2.2 인증서 (Certificate) 구조

2.2.1 인증서 구성 요소

```
pub struct Certificate {

pub header: Header, // 인증서 헤더
```

```
pub signatures: Vec<Signature>, // 검증자들의 서명
}

pub struct Header {
pub author: PublicKey, // 생성자 공개키
pub round: Round, // 라운드 번호
pub epoch: Epoch, // 에포크 번호
pub parents: BTreeSet<CertificateDigest>, // 부모 인증서들
pub payload: IndexMap<BatchDigest, WorkerId>, // 배치 참조
}
```

2.2.2 인증서 생성 과정

1. **헤더 생성**: Primary가 배치 다이제스트와 부모 참조로 헤더 생성
2. **서명 수집**: 다른 노드들로부터 헤더에 대한 서명 수집
3. **임계값 확인**: f+1개 이상의 서명 확보 시 인증서 완성
4. **DAG 삽입**: 완성된 인증서를 로컬 DAG에 추가

2.3 DAG 관리 시스템

2.3.1 InnerDag 구조

```
struct InnerDag {
rx_primary: metered_channel::Receiver<Certificate>,
```

```
rx_commands: Receiver<DagCommand>,
dag: NodeDag<Certificate>,
vertices: RwLock<BTreeMap<(PublicKey, Round), CertificateDigest>>,
metrics: Arc<ConsensusMetrics>,
}
```

2.3.2 주요 기능

- **삽입 (Insert)**: 새로운 인증서를 DAG에 추가
- **조회 (Contains)**: 특정 인증서의 존재 여부 확인
- **인과관계 읽기 (ReadCausal)**: 특정 인증서부터 시작하는 BFS 순회
- **제거 (Remove)**: 메모리 회수를 위한 인증서 압축

2.4 메모리 관리

2.4.1 가비지 컬렉션

```
pub fn update(&mut self, certificate: &Certificate, gc_depth: Round) {
// 최신 커밋 라운드 업데이트
self.last_committed_round = last_committed_round;
// GC 깊이를 초과한 인증서 제거
self.dag.retain(|r, _| r + gc_depth >= last_committed_round);
}
```

2.4.2 압축 메커니즘

- **Mark-and-Sweep**: 참조되지 않는 인증서 표시 후 제거
- **Weak Reference**: 압축된 노드는 약한 참조로 유지
- **Secondary Index**: (PublicKey, Round) → CertificateDigest 매핑 유지

2.5 DAG 순회 알고리즘

2.5.1 너비 우선 탐색 (BFS)

```
fn read_causal(&self, start: CertificateDigest)
-> Result<impl Iterator<Item = CertificateDigest>, ValidatorDagError> {
let bft = self.dag.bft(start)?;
Ok(bft.map(|node_ref| node_ref.value().digest()))
}
```

2.5.2 인과관계 추적

- **Forward Traversal**: 자식 노드들로의 순회
- **Backward Traversal**: 부모 노드들로의 역순회
- **Causal History**: 특정 인증서의 전체 의존성 체인

2.6 동시성 제어

2.6.1 읽기-쓰기 락

```
vertices: RwLock<BTreeMap<(PublicKey, Round), CertificateDigest>>
```

2.6.2 비동기 명령 처리

- **Command Pattern**: 모든 DAG 연산을 명령으로 캡슐화
- **Actor Model**: 단일 스레드에서 순차적 처리
- **Oneshot Channel**: 명령 결과 반환

2.7 성능 최적화

2.7.1 메트릭 수집

```
fn  update_metrics(&self) {
self.metrics.external_consensus_dag_vertices_elements
.set(vertices.len() as i64);
self.metrics.external_consensus_dag_size
.set(self.dag.size() as i64);
}
```

2.7.2 배치 처리

- **Bulk Operations**: 여러 인증서 동시 처리
- **Lazy Evaluation**: 필요시에만 계산 수행
- **Cache Locality**: 관련 데이터의 지역성 최적화

이 DAG 구조는 Narwhal의 핵심으로, 높은 처리량과 낮은 지연시간을 달성하는 기반이 됩니다.